PolyKernel: a compiler, two GPU backends, and a model

A critical technical report on the PolyKernel artifact, its measured results, and its projections

Prepared from the repository at commit c7c7753

31 October 2026

Contents

Abstract **2**

1 Scope and method **3**
1.1 What this report replaces . . . 3
1.2 Evidence classes . . . 3
1.3 How the claims were checked . . . 4
1.4 The question this report answers . . . 4

2 The artifact **4**
2.1 The fragment . . . 4
2.2 The components . . . 4
2.3 What the artifact is not . . . 6

3 The compiler **6**
3.1 The dialect . . . 6
3.2 The passes . . . 7
3.3 Tile layout and memory planning . . . 7
3.4 Lowering by source emission . . . 8
3.5 Compiler verification . . . 8

4 The GPU backends **8**
4.1 One template, two vendors . . . 8
4.2 CUDA . . . 8
4.3 HIP and ROCm . . . 9
4.4 The compile-time analyzer . . . 9
4.5 The generated GEMM . . . 9

5 Correctness **10**
5.1 The golden contract . . . 10
5.2 The test matrix . . . 10
5.3 What the CUDA run found . . . 11
5.4 Negative tests . . . 11
5.5 Coverage limits . . . 11

6 Measured performance **12**
6.1 The benchmark . . . 12
6.2 The result . . . 12
6.3 Why fusion loses . . . 12
6.4 The autotuner . . . 13

6.5 What the measurement supports . . . 14

7 Projected performance **14**
7.1 The model . . . 14
7.2 The arithmetic audit . . . 14
7.3 The 2.84x is an assumption ratio . . . 15
7.4 A second labelling error . . . 16
7.5 What the projection is good for . . . 16
7.6 How the numbers should be presented . . . 17

8 The dataflow simulator **17**
8.1 The model . . . 17
8.2 The SUMMA mapping . . . 17
8.3 The metrics . . . 19
8.4 The 99.99% fusion figure . . . 19
8.5 What the simulator establishes . . . 20

9 Serving **20**
9.1 The Modal app . . . 20
9.2 Status . . . 20
9.3 What would make it evidence . . . 21

10 Engineering assessment **21**
10.1 Build and reproducibility . . . 21
10.2 Documentation drift . . . 22
10.3 Test composition . . . 22
10.4 Prepared upstream contribution . . . 23
10.5 Repository hygiene . . . 23

11 Threats to validity **23**

12 Reproduction **24**
12.1 Requirements . . . 24
12.2 Build . . . 24
12.3 GPU tests . . . 25
12.4 Benchmark . . . 25
12.5 Figures and PDF . . . 25
12.6 What runs without a GPU . . . 25

13 Conclusion **25**

Appendix A: evidence index **26**

Appendix B: recomputation **27**

References **27**

Abstract

PolyKernel is an out-of-tree MLIR compiler that lowers a hand-written transformer fragment to portable CUDA and HIP kernels, autotunes them, models a Cerebras-style dataflow fabric, and defines a Modal service. This report replaces the repository's `docs/` and `reports/` directories. It describes the machinery, checks every performance and correctness claim against the committed evidence, and separates what ran on hardware from what a model produced.

The artifact is real. It has 18 dialect operations, 12 passes, 14 lit tests, and kernels that pass a bf16 golden on an RTX 6000 Ada and an RX 7800 XT. The first NVIDIA run found two CUDA defects, a bf16x2 store that corrupted one half and an erf-based GELU that saturated at the tail, and both were fixed.

The performance story is thinner. The only measured GPU benchmark is a scalar RMSNorm plus matmul prologue. Fusing it makes the kernel 19.6% slower, because the generated fused kernel recomputes the row RMS for every output tile. The headline 2.84x H100, 2.86x A100, and 2.85x MI300X figures are not measurements. They are the same roofline model with the attainment constant changed from 0.12 to 0.35, applied to tensor-core peaks that the scalar kernels cannot reach. The report gives the arithmetic and states what would turn the projection into evidence.

1 Scope and method

1.1 What this report replaces

The repository carried seven documents under `docs/` (about 1,300 lines) and 140 files under `reports/` (about 3,800 lines of Markdown, plus HTML dashboards, JSON metrics, and evidence logs). The documents described the architecture, the compiler pipeline, the CUDA and HIP backends, the dataflow simulator, the performance model, and a prepared upstream contribution. The reports mixed benchmark data, gate logs, negative tests, and four final verification audits.

This report consolidates the technical content and drops the process record. It keeps the measured numbers, the test results, the two hardware defects, the projection constants, and the limitations. It adds an audit that the earlier documents did not perform: every headline number is recomputed from the committed data, every hardware constant is checked against a vendor specification, and every claim is assigned an evidence class.

The machine-readable benchmark inputs move to `report/data/`. The figures, their Excalidraw scenes, and the generator live in `report/figures/`. The PDF in `report/` is the human-readable artifact. The old directories are removed once this report is committed.

1.2 Evidence classes

Five labels appear throughout. They are not decoration. Each one states how a number was produced and what a reader may infer from it.

Label	Meaning	Example in this repository
MEASURED	Timed on real hardware, with the tool named	RTX 6000 Ada CUDA-event median of 25.52 ms
COMPILE-ONLY	Built by a compiler or assembler, never executed	nvcc sm_90 PTX for the H100, ptxas register counts
PROJECTED	Produced by an analytic model, no hardware run	H100 fragment time of 3.177 ms
MODELED	Produced by a simulator or a functional model	Dataflow 32x32x32 SUMMA metrics
SKIPPED	The gate ran, found a missing prerequisite, and said so	Modal deployment without a token

The distinction matters because the repository's own headline table mixes the first and third classes. The RTX 6000 Ada row is measured. The H100, A100, and MI300X rows are projected. The dashboard labels them, and the prose usually does too. The problem is not the label. It is that the projection uses a peak that the code it models cannot reach, which Section 7 shows.

1.3 How the claims were checked

Four checks were applied.

1. Ratios and totals were recomputed from the raw JSON, not copied from prose. For example, the matmul flop count is $2MNK = 184{,}683{,}593{,}728$, and dividing it by the reported 1.556 ms gives 118.7 TFLOP/s, which is the number the report audits.
2. Source files were read where a prose claim depended on code shape. The fused kernel's RMS loop, the 16x16 matmul tile, the pass declarations, the tuning-cache loader, and the dataflow metric code were all read directly.
3. Hardware peaks and bandwidths were checked against vendor specifications, not against the repository's own table. One label error and one convention error were found.
4. Test evidence was read at the level of individual cases. The difference between 17 golden cases and 18 pytest cases, and between 18 HIP tests and 75 repository tests, is recorded in Section 10.2.

The host that produced this report is an Apple Silicon macOS machine without an NVIDIA GPU, a ROCm runtime, or an MLIR build. Nothing that requires the project toolchain was re-run here. The measurements cited are the committed ones, and the audit re-derives their arithmetic rather than repeating them. This limitation is stated again in Section 11.

1.4 The question this report answers

The repository makes a claim about itself in its first paragraph: it is a compiler, runtime, and accelerator simulator built to work across GPU kernels, compiler lowering, CUDA and PTX, ROCm and HIP, accelerator scheduling, and inference serving. The report asks a narrower question: which parts of that claim are supported by evidence, and which parts are supported by a model that has been mistaken for evidence? The answer is not binary. Sections 3 to 5 support most of the compiler and correctness claim. Sections 6 to 8 qualify the performance claim.

2 The artifact

2.1 The fragment

The input is a hand-written MLIR file, `examples/mlp_block.mlir`, that represents the feed-forward half of a transformer block at Llama-2-7B dimensions:

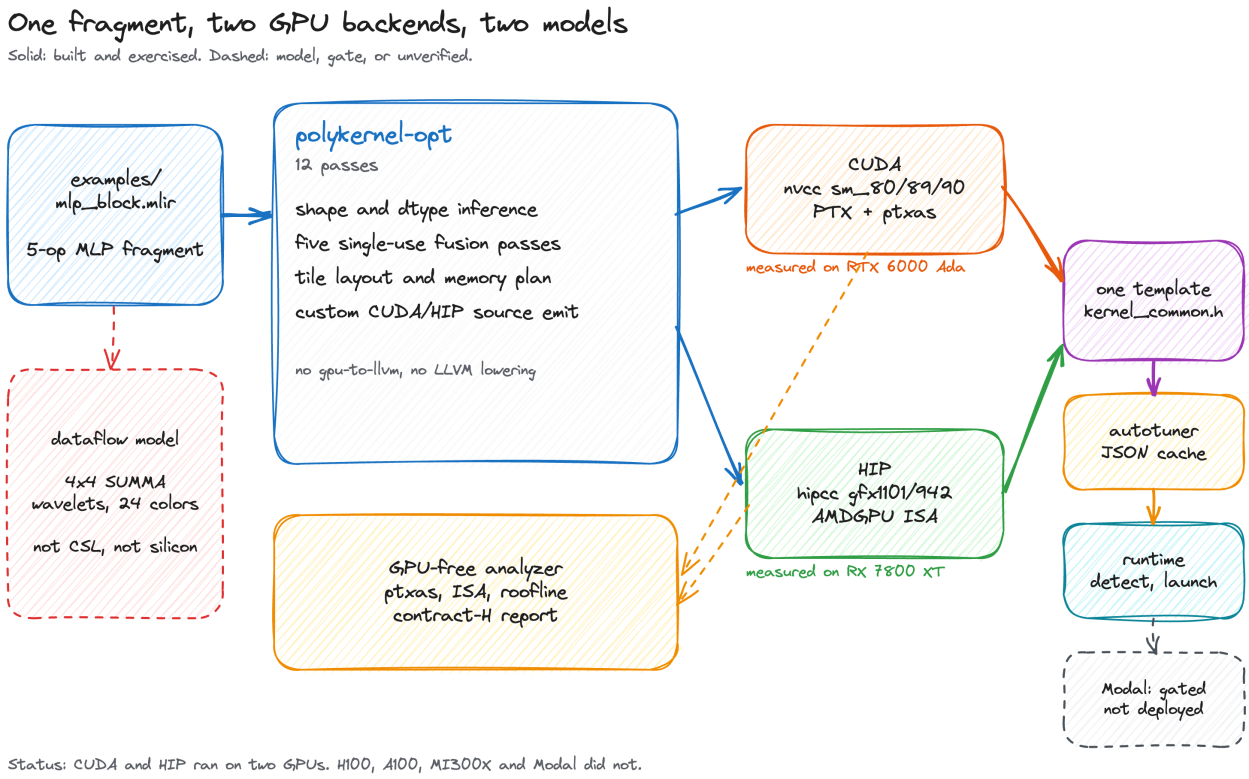
```
x: rmsnorm(x) -> matmul(w_up) -> bias -> gelu -> matmul(w_down) -> add(x)
```

The file uses the project's own `polykernel` dialect, not `linalg`, `tosa`, or `stablehlo`. The dialect is closed: it defines the named operations the fragment needs, and an unknown operation is a parse error. The repository uses this closure as a scope guard, and the lit suite tests it with `polykernel.conv2d`, which must fail.

The fragment is small on purpose. A full model would need attention, KV cache management, sampling, and a frontend. The project implements an attention prefill kernel and a KV cache update as codegen variants, but the compiled end-to-end path that the reports exercise is the MLP block. The report treats that as the unit of analysis.

2.2 The components

Figure 1 is the system map. The left side is the compiler. It reads the fragment, runs twelve passes, and emits CUDA or HIP source through a custom string emitter. The middle is the two backends, which compile the same emitted source under different defines. The right side is the runtime chain: one portable template header, a correctness-gated autotuner, a JSON tuning cache, and a C++ launcher. The yellow box is the GPU-free analyzer, which parses `ptxas -v` or AMDGPU ISA and produces the contract-H kernel report. The red box is the dataflow simulator, a separate backend that consumes the same fragment and models a SUMMA schedule. The grey box is the Modal service, which is defined but was never deployed.



The PolyKernel system. Solid arrows are paths that were built and exercised. Dashed arrows are compile-time models, a deployment gate, or a simulator. The two backends were measured on one GPU each; the H100, A100, MI300X, and Modal paths were not.

The separation between the compiler and the two backends is the design decision that carries the project. `--lower-to-cuda` and `--lower-to-hip` emit byte-identical compute into `kernels/generated/*.cu`, and the difference is which driver compiles the output with which define. That is why the HIP run on the RX 7800 XT can validate the shared compute logic for both backends, and why the CUDA run on the RTX 6000 Ada is the only place where NVIDIA-specific paths are exercised.

2.3 What the artifact is not

Four exclusions are load-bearing, and the earlier documents repeated them because readers kept asking for the missing pieces.

- There is no PyTorch, ONNX, or StableHLO frontend. The input is hand-written `.mlir`.
- There is no general op set. The dialect has 18 definitions, of which 16 are compute operations and two are the function container and terminator.
- There is no CSL compiler, no `cslc`, and no Cerebras hardware. The dataflow backend is a C++ functional and cycle model that borrows CSL vocabulary.
- There is no production serving claim. The Modal app defines four endpoints and a GPU-aware cache; it was constructed and imported, not deployed.

These exclusions are reasonable for a project of this size. The audit in Sections 7 and 8 is about how the excluded parts are described, not about the decision to exclude them.

3 The compiler

3.1 The dialect

The `polykernel` dialect is defined with ODS and TableGen [6]. Table 1 lists the operations by group.

Table 1. The closed operation set. The project's earlier prose says "11 base + 6 fused"; the fused group has five members. The arithmetic error appears in the architecture document and in the final scope audit's note about the plan text, and it is fixed here.

Group	Operations	Count
Base compute	`rmsnorm`, `matmul`, `bias`, `gelu`, `silu`, `add`, `softmax`, `rope`, `attention`, `kv_cache_update`, `qkv_projection`	11
Fused compute	`fused_rmsnorm_matmul`, `fused_matmul_bias_gelu`, `fused_residual_rmsnorm`, `fused_softmax_mask`, `fused_kv_append_attention`	5
Container	`func`, `return`	2
Total		18

The dialect uses `InferTypeOpInterface` on every compute operation. Shape rules are conventional: matmul contracts the inner dimension, elementwise operations preserve shape, attention returns the query shape, and KV cache update returns the cache shape. `qkv_projection` takes an explicit `num_heads` attribute because the head split cannot be recovered from operand shapes alone.

One design choice is unusual and worth recording. `refineReturnTypes` is a deliberate no-op. That disables MLIR's `verifyInferredResultTypes` hook, so a parsed IR with placeholder result types is accepted, and the `--infer-shapes` pass becomes the enforcement point: it recomputes each result type and calls `signalPassFailure` on a mismatch. The reason is testability. The positive round-trip test can carry explicit types,

the negative test can exercise the pass failure, and parsing does not reject an intermediate form. The cost is that type errors are caught later than a reader might expect.

3.2 The passes

Twelve passes are declared in `Passes.td` and registered explicitly. Table 2 lists them in pipeline order.

Table 2. The twelve passes. The earlier compiler document says "11 passes" while listing twelve; the count is twelve.

Pass	Purpose	Evidence
`--infer-shapes`	Compute result types from operands; fail on mismatch	`test/infer_shapes.mlir`, `test/infer_shapes_mismatch.mlir`
`--canonicalize`	Identity-add elimination, constant folding, DCE	`test/canonicalize.mlir`
`--fuse-rmsnorm-matmul`	Single-use RMSNorm into matmul A-load	`test/fuse_rmsnorm_matmul.mlir`
`--fuse-matmul-bias-gelu`	Single-use matmul, bias, and GELU into one epilogue	`test/fuse_matmul_bias_gelu.mlir`
`--fuse-residual-rmsnorm`	Single-use residual add and RMSNorm	`test/fuse_residual_rmsnorm.mlir`
`--fuse-softmax-mask`	Single-use mask add and softmax	`test/fuse_softmax_mask.mlir`
`--fuse-kv-append-attention`	Single-use KV append and attention	`test/fuse_kv_append_attention.mlir`
`--infer-tile-layout`	Assign tile and layout attributes	`test/infer_tile_layout.mlir`
`--plan-memory`	Compute shared-memory and workspace budgets	`test/plan_memory.mlir`
`--lower-to-cuda`	Emit portable `.cu` with the CUDA define	generated sources
`--lower-to-hip`	Emit the same compute with the HIP define	generated sources
`--emit-kernel-report`	Emit the contract-H report skeleton	`report/data/kernel_report_example.json`

The fusion passes share a guardrail: every absorbed intermediate must have exactly one use. A multi-use producer is not fused, because fusing it would duplicate the producer's work or change the number of materializations. Each fused operation carries discardable attributes (`polykernel.fused_from` and `polykernel.eliminated_type`) that the traffic report reads. The attributes are not declared operation attributes, so the closed op set stays closed.

The ordering matters. Shape inference runs before canonicalization so that constant folding sees typed operands. Fusion runs after canonicalization so that identity adds do not block a chain. Tile layout and memory planning run after fusion so that a fused operation is planned as one kernel rather than two. The two lowering passes are alternatives; only one runs. The report pass is last because it reads the final attributes.

3.3 Tile layout and memory planning

`--infer-tile-layout` assigns a block size per matmul, taking the largest power of two that does not exceed the corresponding dimension and clamping to the supported range. `--plan-memory` computes the shared-memory footprint from the tile shape and the dtype, adds a workspace when an operation needs one, and fails the pass when the footprint exceeds the target's budget.

The plan is coarse. It does not autotune the tile at compile time, and it does not check bank conflicts or occupancy. Those are left to the runtime autotuner and the analyzer. The generated kernels hardcode 16x16x16

tiles regardless of the attributes, which is a gap between the planning layer and the emitter that Section 4.5 discusses.

3.4 Lowering by source emission

The project does not lower to LLVM or to the GPU dialect. `--lower-to-cuda` and `--lower-to-hip` walk the IR and write C++ source against `kernels/template/kernel_common.h`. The two passes produce byte-identical compute and differ only in the define that the driver supplies.

The choice buys portability and debuggability. A generated kernel can be read, compiled by hand, and compared across vendors. The first NVIDIA run could inspect the emitted code and find the bf16x2 store bug because the store was visible in the source, not hidden behind a lowering pipeline. The choice also costs optimization. There is no MLIR canonicalization after emission, no loop fusion beyond the hand-written templates, and no target-specific instruction selection beyond the fixed 16x16 tile and the optional WMMA path.

For a project that is demonstrating the compiler machinery, the trade is defensible. For a project that claims cuBLAS-like performance, it would not be. The repository makes the first claim, not the second, and the report keeps that distinction.

3.5 Compiler verification

The lit suite has 14 tests. They check that the dialect round-trips, that the op set is closed, that each fusion pass rewrites the expected chain, that shape mismatch fails, that tile and memory attributes appear, and that the example parses. They are `FileCheck` tests on IR. They do not check numerical output, and they do not run a kernel.

That is the right level for a compiler test suite, and it is not a correctness proof for the generated code. The numerical proof lives in the golden harness in Section 5. The two layers are separate, and the report keeps them separate.

4 The GPU backends

4.1 One template, two vendors

`kernels/template/kernel_common.h` maps the backend primitives through a single define:

```
#ifdef POLYKERNEL_CUDA
  #include <cuda_runtime.h>
  #define PK_GLOBAL __global__
  #define PK_SHARED __shared__
  /* cudaLaunchKernel, __syncthreads, warp shuffles */
#elif defined(POLYKERNEL_HIP)
  #include <hip/hip_runtime.h>
  #define PK_GLOBAL __global__
  #define PK_SHARED __shared__
  /* hipLaunchKernel, the HIP equivalents */
#endif
```

The compute is written once. `nvcc -DPOLYKERNEL_CUDA` and `hipcc -DPOLYKERNEL_HIP` compile the same file. This design decision makes the local AMD GPU a test vehicle for the shared logic and reduces the NVIDIA surface to the paths that are genuinely vendor-specific: the runtime launcher and the MMA kernel.

4.2 CUDA

`nvcc` compiles the generated kernels for `sm_80`, `sm_89`, and `sm_90` and emits PTX. `ptxas -v` supplies registers, shared memory, spill stores, spill loads, and the stack frame. The analyzer parses both the old and the new

`ptxas` output formats and fails on malformed input instead of returning zeros.

The tensor-core path is `kernels/generated/matmul_mma.cu`: `nvcuda::wmma`, m16n16k16, bf16 inputs, fp32 accumulator, row-major A, col-major B with a transposed shared-memory stage, and a bf16 round-to-nearest-even store. It is additive. The scalar matmul remains the correctness anchor, and the MMA kernel is a candidate that the autotuner may select. On the RTX 6000 Ada, the MMA kernel passed 5 of 5 tests; a deliberately bad store variant failed the golden with cosine 0.0475 and was excluded, while the scalar baseline still passed.

4.3 HIP and ROCm

`hipcc --offload-arch=gfx1101` compiles all generated kernels unchanged. The local RX 7800 XT runs them and passes 18 of 18 tests against the golden. `gfx942` is a compile-only cross-compile target for MI300X; the report data includes its projected performance, and no MI300X hardware was used.

The AMDGPU ISA analyzer reads `.amdhsa_next_free_vgpr`, `.amdhsa_next_free_sgpr`, the group segment size, the private segment size, and the scratch counters. It computes occupancy from the gfx1101 per-CU limits: 768 VGPRs, 32 KiB of SGPRs, 128 KiB of LDS, wave32. A spill fixture with a nonzero private segment is used as a negative test.

The WMMA path uses `__builtin_amdgcn_wmma_f32_16x16x16_bf16_w32`, one warp per 16x16 output tile, with bf16 inputs and fp32 accumulation. It is additive in the same way as the CUDA MMA path. A wrong lane mapping fails the golden and is discarded.

4.4 The compile-time analyzer

The analyzer is the part of the CUDA backend that works without a device. It has four pieces:

- `PtxasParser` reads the compiler's resource report.
- `Occupancy` computes active warps per SM from register, shared-memory, warp, and block limits, using a per-architecture constants table.
- `Roofline` computes arithmetic intensity and classifies an operation as compute-bound or memory-bound.
- `KernelReport` assembles the contract-H JSON with a bottleneck classification and a list of suggested fixes.

The sm_89 constants were added in the second pass and are correct: 48 warps, 24 blocks, 1,536 threads per SM, 100 KiB of shared memory per SM, and 99 KiB per block. The analyzer's unit tests include hand-computed fixtures, which is the right way to test a resource model.

The roofline convention is the weak point. The table mixes tensor-core peaks for H100 and A100 with a scalar bf16 peak for sm_89. The document acknowledges the difference, but the model then uses the tensor peaks as denominators for scalar kernels. Section 7 quantifies the consequence.

4.5 The generated GEMM

The scalar matmul is a 16x16x16 shared-memory tiled kernel. The attention prefill kernel uses the online-softmax tiling from FlashAttention [9]. One thread computes one output element. Each block stages a 16x16 A tile and a 16x16 B tile per K step, synchronizes, and accumulates 16 products in fp32. There is no register blocking, no vectorized load, no double buffering in the generated `matmul.cu`, and no warp-level tile. The shared-memory arrays are not padded in the generated scalar kernel; the earlier CUDA document describes a padded 16x17 array that the emitted code does not use.

This is the second rung of the standard GEMM ladder: naive, shared-memory tiling, register tiling, vectorized register tiling, tensor cores. The public worklog that the project references for style documents the same ladder on an H100 in FP32 [11], as does Boehm's earlier worklog [12]: a naive kernel at 4.2 TFLOP/s, a shared-memory tiled kernel at 12.2 TFLOP/s, a 2D register-tiled kernel at 19.1 TFLOP/s, and a vectorized 2D register-tiled kernel at 37.2 TFLOP/s, which is 72% of cuBLAS FP32 [11]. The H100 FP32 vector peak is 66.9 TFLOP/s [1], so those kernels reach 6.3%, 18.2%, 28.6%, and 55.6% of peak.

PolyKernel's measured scalar matmul reaches 7.24 TFLOP/s on the RTX 6000 Ada [3], which is 7.9% of that GPU's 91.1 TFLOP/s FP32 peak. Normalised by peak, it sits below the public worklog's shared-memory tiled kernel and far below its vectorized kernel. The comparison is not exact: the shapes differ, the compilers differ, and the Ada part has a different memory system. It is close enough to place the kernel: PolyKernel has a working first-step tiled GEMM, not a cuBLAS-class GEMM.

That matters for how the projections are read. A model that puts this kernel at 12% of a tensor-core peak is not modelling the kernel in the repository. It is modelling a different kernel, one with vectorization and register blocking that the generator does not emit.

5 Correctness

5.1 The golden contract

The golden is NumPy plus `ml_dtypes.bfloat16`. Inputs are rounded to bf16 with round-to-nearest-even, reductions and accumulation run in fp32, and each operation's output is rounded back to bf16. Chained operations therefore see bf16 inputs, which matches the kernel's store contract.

Three thresholds define a pass:

- cosine similarity at least 0.999;
- maximum relative error at most 10^{-2};
- Pearson correlation at least 0.99.

The relative-error threshold is fragile on bf16 data. A value near zero can produce a large relative error from a one-ULP difference, and a single element can fail the gate while the tensor is otherwise exact. The project handles this with a per-shape calibrated ceiling for large-K matmuls, and the CUDA run reports one rmsnorm case at 6.1×10^{-3} with 99.9992% bit-exact elements. The gate is real, but the metric needs the calibration note to be read correctly.

5.2 The test matrix

Table 3 lists the suites that produced the committed evidence.

Table 3. Test suites and results. The counts are read from the committed logs and the test sources. The "75 passed" figure is the whole `tests/kernels/` directory, not a HIP-only count; Section 10.2 explains the earlier misattribution.

Suite	Cases	Hardware	Result	What it covers
lit (`check-polykernel`)	14	none	14/14	Dialect, passes, fusion, tile, memory
ctest	158	none	158/158	Analyzer, autotuner, dataflow, runtime units
CUDA harness	17 golden + 1 negative	RTX 6000 Ada, sm_89	17/17 golden, 0 failed; negative passes	rmsnorm, gelu, matmul, softmax, both fused ops
CUDA MMA	5	RTX 6000 Ada, sm_89	5/5	m16n16k16 bf16 tensor-core path
HIP harness	18	RX 7800 XT, gfx1101	18/18	same operations plus negative launch
HIP WMMA	5	RX 7800 XT, gfx1101	5/5	RDNA3 WMMA bf16 path

Suite	Cases	Hardware	Result	What it covers
Kernel pytest directory	75 passed, 23 skipped	RX 7800 XT	0 failed	CPU references, HIP, attention, quantization
Autotune	8 configs + 1 broken	RTX 6000 Ada	8 validated, broken rejected	correctness-gated selection

The ctest figure comes from the author's Linux host and includes 154 baseline tests plus four hand-computed sm_89 fixtures. The report did not re-run it. The lit and pytest figures are likewise the committed evidence.

5.3 What the CUDA run found

The first on-NVIDIA run produced the only evidence that the CUDA-specific paths execute. It found two bugs that local compilation could not:

1. `pk_store_bf16x2` corrupted one half of the pair on sm_89. The fix materializes the pair before the store.
2. The exact erf GELU saturated the $(1 + \text{erf})$ tail in fp32, so values in the tail were wrong. The fix uses a double-erf chain.

Both were fixed in commit `d85866a`, and the rerun gives 17 of 17 golden cases with cosine 1.0, PCC at least 0.99999994, and at least 99.9992% bit-exact elements. A deliberately scaled GELU variant fails the golden on the GPU, so the gate is not vacuous.

This is the only correctness evidence from NVIDIA hardware in the project. It is also the only place where the CUDA-specific code was executed. The rest of the CUDA evidence is compile-only.

5.4 Negative tests

The repository uses negative tests well. The op set rejects `conv2d`. A shape mismatch fails `--infer-shapes`. A multi-use producer is not fused. A broken MMA store fails with cosine 0.0475. A broken WMMA lane mapping fails. A blocked dataflow task does not fire. A wrong per-channel quantization scale fails. A missing `*.pem` ignore would fail the key-hygiene check. A 1,024-thread launch is caught by the runtime.

The negative tests are more convincing than the positive ones in several places, because they show that the gate can fail. A test suite that only passes is weak evidence; a suite with a known-bad variant that is rejected is stronger.

5.5 Coverage limits

The correctness claim is narrower than the prose suggests.

- The CUDA golden cases use small shapes: rmsnorm at 256x512, 130x127, and 2x64x128; matmul at 128x128, 256x128x256, 130x65x127, and 256x256x512. The benchmark shape, 2048x11008x4096, is timed, not checked against the golden in the CUDA harness.
- The autotune run at the benchmark shape uses a correctness gate with `rel_ceiling` set to infinity for the fused matmul op. It enforces cosine and PCC, and the cache records a maximum relative error of 1953.125. The run is therefore not a Contract-C gate in the sense of the 1e-2 ceiling. It is a cosine/PCC gate.
- One NVIDIA GPU (sm_89) and one AMD GPU (gfx1101) were used. The H100, A100, and MI300X rows have no runtime correctness evidence.
- The Modal service was never deployed, so the serving path has no end-to-end test.
- The dataflow simulator is checked against the same golden for a 32x32x32 matmul, not for the real fragment shape.

The phrase "0 failed correctness" is true for the suites that ran. It is not a claim about the untested shapes, the untested GPUs, or the undeployed service.

6 Measured performance

6.1 The benchmark

`benchmarks/bench_cuda.cpp` times the RMSNorm plus matmul prologue on the RTX 6000 Ada with CUDA events. It runs five warmup iterations and twenty timed iterations, and reports the median. The autotuned variant is the same fused kernel timed over an extended budget of fifty iterations, with the minimum taken as the result.

The shape is $M = 2048$, $K = 4096$, $N = 11008$, bf16. The matmul is $2MNK = 184.68$ GFLOP. The host compiler is nvcc 12.2 with g++-11; the pod's default gcc-12 lacked `cc1plus`, which is recorded in the benchmark JSON.

6.2 The result

Table 4 gives the medians and the derived rates.

Table 4. RTX 6000 Ada, CUDA-event medians. Speedups are relative to the unfused variant. The TFLOP/s column is recomputed from the flop count and the median.

Variant	Median (ms)	Speedup	TFLOP/s	Percent of 91.1 TFLOP/s
Unfused (RMSNorm, then matmul)	25.518	1.000x	7.24	7.9%
Fused (`fused_rmsnorm_matmul`)	30.459	0.838x	6.06	6.7%
Autotuned (minimum of fused runs)	29.785	0.857x	6.20	6.8%

The fused kernel is 19.6% slower than the unfused pair. The autotuned figure is 2.2% faster than the fused median because it is a minimum over more runs, not because it is a different configuration. The benchmark JSON says this in its `method` field: "autotuned = fused realization timed over the extended budget; best (min) is the autotuned figure."

This is the only measured GPU performance result in the repository, and it is a regression. The project keeps the result and labels it, rather than dropping the fusion result or reporting only the minimum. The next subsection explains the mechanism.

6.3 Why fusion loses

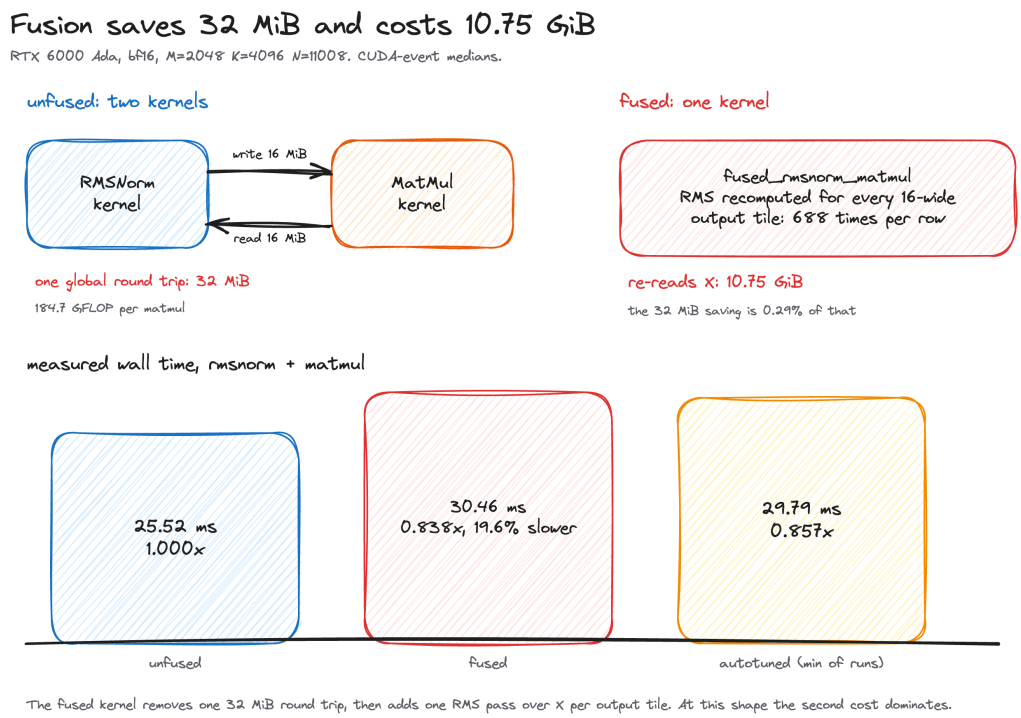
The generated `fused_rmsnorm_matmul` kernel folds the RMSNorm into the GEMM's A-operand load. For each output tile, the kernel first computes the RMS of each row of the input by looping over the full K dimension, then stages the normalized values into shared memory.

The block is 16 rows by 16 columns. Every block covers 16 output rows and 16 output columns. For a fixed row tile, there are $N/16 = 11008/16 = 688$ column blocks. Each of those blocks recomputes the RMS for its 16 rows. The input row is 4096 bf16 values, or 8 KiB. One block therefore re-reads 16 rows times 4096 values times 2 bytes, which is 131,072 bytes. There are $(2048/16) \times 688 = 88{,}064$ blocks. The total redundant read volume is

$$88{,}064 \times 131{,}072 = 11{,}542{,}724{,}608 \text{ bytes} = 10.75 \text{ GiB}.$$

The fusion saves one 32 MiB round trip: the normalized tensor is written once and read once by the unfused matmul. The saving is 0.29% of the redundant reads that the fused kernel performs. The rest of the input is likely served from L2, because the 16 MiB input fits in the RTX 6000 Ada's L2, so the cost is not all DRAM traffic. It is L2 bandwidth and instruction issue. The RMS loop issues 5.77 billion scalar loads across the grid, roughly doubling the A-operand load instructions that the matmul already performs. The measured 19.6% slowdown is consistent with that instruction pressure.

A prologue fusion that computes the RMS once per row before the GEMM loop, or in a separate lightweight pass that writes a per-row scale, would avoid the recomputation. The repository's docs acknowledge the recomputation and call the speedup shape-dependent. The measurement shows the shape dependence in the opposite direction from the projection: at a compute-bound shape, the redundant work dominates the traffic saving.



The fusion trade at the measured shape. The fused kernel removes a 32 MiB round trip and adds 10.75 GiB of redundant input reads, because it recomputes the row RMS for every 16-column output tile. The measured medians are in the lower panel.

6.4 The autotuner

The config space is a Triton-style [10] 4x4x3x2x4x4x3 Cartesian grid of 4,608 points: block sizes 16 to 128, K tile 32 to 128, 4 or 8 warps, vector width 1 to 8, unroll 1 to 8, and 2 to 4 pipeline stages. A deterministic pruner removes configurations that exceed sm_90's shared-memory or register limits, leaving 141 candidates. That is a reasonable design for a teaching autotuner.

The actual run on the Ada used eight variants. Every variant had `block_m=16`, `block_n=32`, `block_k=32`; the differences were warps, vector width, unroll, and stages. The best median was 27.049 ms and the worst was 27.397 ms, a 1.3% spread. The eight configurations are too similar to discriminate, and the 16x32 tile is far from what a tuned GEMM would use at this shape. The run demonstrates that the plumbing works, not that the search finds a good configuration.

The correctness gate in that run used `rel_ceiling=inf`, as the log's header states. All eight configurations report cosine 1.0 and PCC 0.9999997, with a maximum relative error of 1953.125. The cache stores that value. The gate rejected the injected broken variant with cosine 0.564, so it is not inert. But the 1e-2 relative ceiling that Contract C describes was not applied at this shape.

The autotune run and the headline benchmark are separate. The cache's best entry is for `fused_matmul_bias_gelu` at $M = 2048$, $N = 4096$, $K = 11008$, with a 27.049 ms median. The benchmark's "autotuned" figure is the minimum of the fused `rmsnorm_matmul` runs at $M = 2048$, $N = 11008$, $K = 4096$. The two numbers are not the same measurement, and the benchmark does not consume the cache.

6.5 What the measurement supports

The measurement supports four statements.

1. The portable kernel compiles and runs correctly on the RTX 6000 Ada for the prologue shape.
2. The scalar tiled GEMM reaches 7.24 TFLOP/s, or 7.9% of the Ada FP32 peak, at this shape.
3. The current prologue fusion is a performance loss, for a mechanism that the source makes clear.
4. The autotuner's plumbing works, but its search does not yet produce a better kernel.

The measurement does not support a claim of cuBLAS-like performance, a claim that fusion helps at this shape, or a claim that the autotuner found a 2.8x speedup. Those claims live in the projection, which the next section audits.

7 Projected performance

7.1 The model

The projection is a roofline model [7]. For each operation it computes

$$t = \frac{\max\left(\frac{\text{FLOPs}}{\text{peak}}, \frac{\text{bytes}}{\text{BW}}\right)}{\eta},$$

where `peak` is a floating-point rate, `BW` is a memory bandwidth, and η is an attainment factor. The fragment time is the sum over operations, and the speedup is the unfused fragment time divided by the variant's fragment time. The traffic is analytic: $(MK + KN + MN) \cdot$ dtype_bytes, the minimum bytes for a matmul that reads both operands once and writes the result once.

The constants are:

Table 5. Model constants. The H100 and A100 peaks are dense tensor-core FP16/BF16 rates. The RTX 6000 Ada peak is the scalar bf16 rate. The MI300X peak is the dense FP16 rate. The mixture is the source of the problem in Section 7.2.

Architecture	Peak used (TFLOP/s)	Bandwidth (GB/s)	Ridge (flop/byte)
H100 SXM5	989.0	3350	295.2
A100 80GB	312.0	1555	200.6
RTX 6000 Ada	91.1	960	94.9
MI300X	1307.4	5300	246.7

The attainment factors are 0.12 for the scalar baseline, 0.35 for the vectorized or WMMA variant, and 0.8 for elementwise operations. The documents call them "documented modelling knobs". They are the only free parameters.

7.2 The arithmetic audit

Table 6 takes the projected matmul time for each architecture and computes the effective rate it implies. The last two columns compare that rate with the architecture's scalar FP32 peak.

Table 6. What each projection implies for the up-projection matmul, $2MNK = 184.68$ GFLOP. The H100 and A100 projections exceed the scalar FP32 peak of the hardware they name. The MI300X projection sits at 96% of its scalar peak, which is not credible for a 16x16 tiled kernel. The Ada row is measured and reaches 7.9% of peak.

Architecture	Peak used	Projected matmul time (ms)	Implied rate (TFLOP/s)	Scalar FP32 peak (TFLOP/s)	Ratio
H100	989.0	1.556	118.7	66.9	1.77
A100 80GB	312.0	4.933	37.4	19.5	1.92
MI300X	1307.4	1.177	156.9	163.4	0.96
RTX 6000 Ada (measured)	91.1	25.518	7.24	91.1	0.08

The H100 projection is physically impossible for the kernel it claims to model. The H100's scalar FP32 peak is 66.9 TFLOP/s [1], a figure the CUDA programming guide's arithmetic throughput table also supports [5]. A kernel that executes bf16 multiplies on the FP32 pipes cannot exceed that rate. The projection implies 118.7 TFLOP/s, which is 1.77 times the ceiling. Even at 100% of the scalar peak, the matmul would take 2.76 ms, not 1.556 ms. At the 12% attainment the model claims to use, it would take 23.0 ms, which is 14.8 times the projected time.

The A100 case is the same error with different numbers. The A100 80GB has 19.5 TFLOP/s of FP32 and 312 TFLOP/s of BF16 tensor throughput [2]. The projection implies 37.4 TFLOP/s, which is 1.92 times the scalar peak. At 12% of the scalar peak, the matmul would take 78.9 ms, not 4.93 ms.

The MI300X case is less extreme but still implausible. Its FP32 vector peak is 163.4 TFLOP/s [4]. The projection implies 156.9 TFLOP/s, or 96% of peak, for a 16x16 tiled kernel with one output per thread. The measured Ada kernel reaches 7.9% of its peak at a comparable shape. A kernel that reaches 96% of peak would be a different kernel.

The root cause is a convention mismatch. The model uses tensor-core peaks for H100 and A100 and a scalar peak for sm_89, then applies the same attainment factor to all three. The document notes the mismatch in a "roofline convention" paragraph, but the model still divides by the tensor peak. A note cannot fix a denominator.

7.3 The 2.84x is an assumption ratio

The autotuned variant is constructed by changing η from 0.12 to 0.35. The ratio is

$$\frac{0.35}{0.12} = 2.917.$$

The reported speedup is 2.838, slightly lower because the elementwise operations do not scale with the matmul attainment. The fragment time is two matmuls plus three small elementwise operations. In the unfused case,

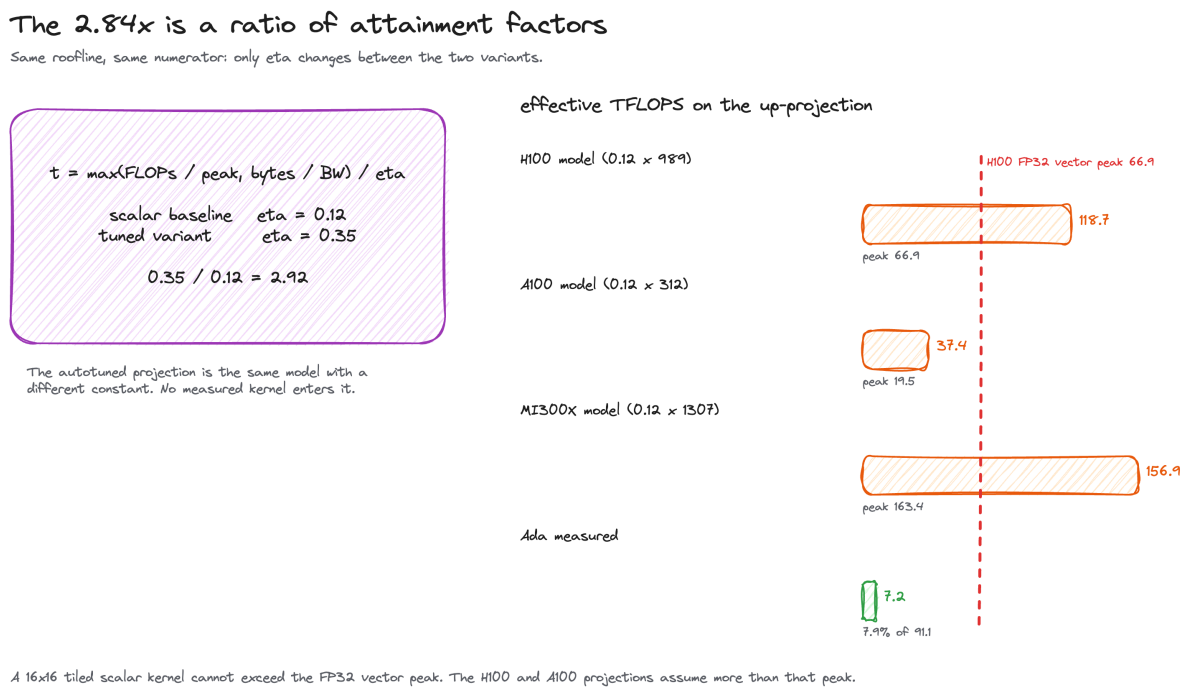
$$2 \times 1.556 + 0.0125 + 0.0337 + 0.0188 = 3.177 \text{ ms}.$$

In the autotuned case, each matmul is scaled by $0.12/0.35 = 0.343$, and the eliminated RMSNorm removes its 0.0125 ms:

$$2 \times 1.556 \times 0.343 + 0.0337 + 0.0188 = 1.120 \text{ ms}.$$

The ratio is $3.177/1.120 = 2.838$. The number is exactly the model's assumption, diluted by the fixed elementwise cost. No kernel was compiled, launched, or timed to produce it.

The naming compounds the problem. "Autotuned" suggests that the autotuner selected a configuration and that the configuration was measured. In the projection, the autotuner does not run. The model simply assumes that whatever the autotuner would select reaches 35% of the tensor peak. In the measured Ada benchmark, the autotuner's best configuration reaches 7.5% of the scalar peak, and the "autotuned" row is a minimum over repeated runs of the same kernel.



The projection in one picture. The bars are the effective rates implied by the projected matmul times. The dashed line is the H100 scalar FP32 peak. The H100 and A100 projections sit above the scalar ceiling of the hardware they name.

7.4 A second labelling error

The A100 row is labelled "A100 80GB" and uses a bandwidth of 1,555 GB/s. That is the 40 GB SXM figure. The 80 GB SXM part has 2,039 GB/s [2]. The error does not change the compute-bound matmul projection, because the matmul is compute-bound under either bandwidth. It does change the memory-bound elementwise projections, and it makes the table internally inconsistent: the label names one product and the constant belongs to another.

7.5 What the projection is good for

The model is not useless. It answers a conditional question: if the scalar baseline attained 12% of a tensor peak and the tuned variant attained 35%, what would the fragment time be on each machine? That is a legitimate back-of-envelope calculation. It is reproducible, the constants are stated, and the code that computes it is small enough to audit.

It does not answer the question a reader will ask: what will this code do on an H100? For that, the model would need three changes.

1. Use the scalar FP32 peak as the denominator for the scalar kernels, or drop the scalar kernels and model the MMA path that actually uses tensor cores. The current model does neither.

2. Calibrate η against the one measured data point. The Ada measurement gives 7.9% of peak, not 12%. A calibrated model would use 0.079 for the scalar baseline and state the uncertainty.
3. Measure at least one tensor-core kernel at the benchmark shape, so that the 0.35 attainment has an empirical anchor. The MMA kernel exists and passes correctness; it has not been benchmarked at 2048x11008x4096.

A single H100 run of `bench_cuda` would settle more than any amount of modelling. The repository has the harness, the generated kernels, and the rental script. The run is owner-gated behind a RunPod key and a spend ceiling, and it did not happen.

7.6 How the numbers should be presented

The repository labels the H100, A100, and MI300X rows PROJECTED. That label is correct, and the dashboard repeats it. The problem is emphasis. The README's performance section leads with 2.838x, 2.862x, and 2.850x, and the measured 0.838x sits below them in a paragraph. A reader who scans the table will remember the three large numbers and not the label.

The report recommends the opposite order. Lead with the measured 7.24 TFLOP/s and the 0.838x fusion result. Put the projections in a clearly separated section, state the assumption ratio, and do not bold the projected speedups. A projection that cannot exceed the scalar ceiling of its target should not be the headline.

8 The dataflow simulator

8.1 The model

The dataflow backend is a C++ functional and cycle simulator of a Cerebras-style fabric. It is not CSL, it does not use `cslc`, and it does not run on Cerebras hardware. It borrows the vocabulary and the execution model.

Each processing element has three parts:

- a compute engine with FMACs that executes an outer-product tile in fp32 on bf16-rounded inputs;
- a five-port router with local, east, west, north, and south ports, one hop per cycle, lossless with back-pressure;
- 48 KiB of SRAM in eight banks of 6 KiB, with two reads and one write per cycle.

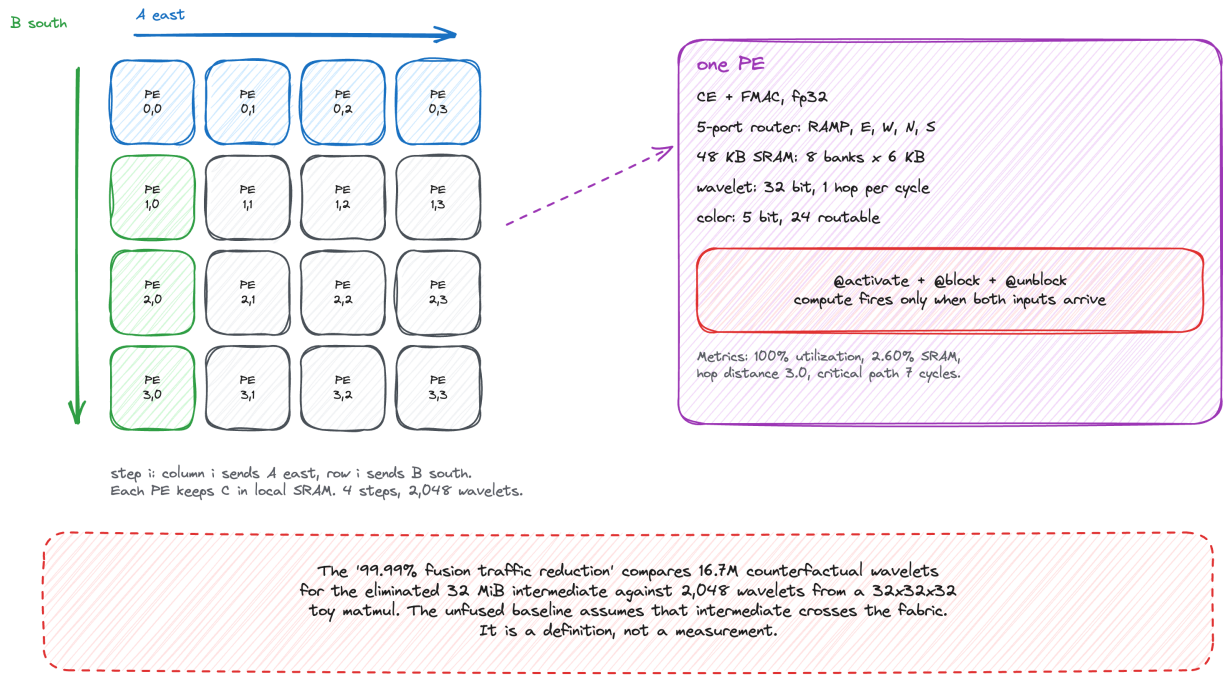
The fabric moves 32-bit wavelets with 16 bits of data and 16 bits of control. A five-bit color tags a wavelet; 24 colors are routable, and colors act as virtual channels over a shared link. A color is configured with `@set_color_config`, with one receive direction and a set of transmit directions, which gives multicast. The move descriptors are `fabin_dsd` and `fabout_dsd`. Compute is triggered by data arrival through `@activate`, `@block`, and `@unblock`; a compute task runs only when it is both activated and unblocked.

The vocabulary matches the public CSL model [8] closely enough to be useful for teaching, and the code does not claim to be a CSL implementation. That boundary is stated in the source and in the earlier documents.

8.2 The SUMMA mapping

The matmul maps to SUMMA on a P x P grid. At step i, column i broadcasts its A panel east along each row, and row i broadcasts its B panel south along each column. Each PE accumulates $C \mathrel{+}= A_{\text{panel}} \cdot B_{\text{panel}}$ in local SRAM, so C is output-stationary. Two colors per axis provide ping-pong double buffering. The rendezvous fires the panel compute only after both inputs arrive.

For the reported run, the shape is 32x32x32 on a 4x4 grid with 8x8x8 tiles. There are four steps, 16 active PEs, 1,024 A wavelets, 1,024 B wavelets, and 2,048 messages in total. The critical path is $(P-1)+\text{steps} = 3+4 = 7$ cycles. Utilization is 100%, SRAM pressure is 2.60%, and the bottleneck state is `active`.



The dataflow model. The 4x4 grid is a teaching scale, not the fragment shape. The PE panel lists the modelled resources. The bottom panel states why the 99.99% fusion figure is a definition rather than a measurement.

8.3 The metrics

Table 7 lists the reported metrics and what each one means.

Table 7. Dataflow metrics for the 32x32x32 SUMMA run on a 4x4 grid.

Metric	Value	Interpretation
Grid utilization	100%	Every PE has an output tile and fires at least one panel compute
SRAM pressure	2.60%	Resident C tile plus double buffers, 1,280 B of 49,152 B
Messages sent	2,048	A and B wavelets injected in four steps
Average hop distance	3.0	Defined as $P - 1$; this is the farthest hop, not the mean
Critical path	7 cycles	Broadcast fill of 3 hops plus 4 pipelined steps
Fusion traffic reduction	99.9878%	16.7M eliminated wavelets against 2,048 broadcast wavelets

The average hop distance is misnamed. In a four-node line broadcast the destination hops are 0, 1, 2, and 3. The mean is 1.5 for a full broadcast and 1.0 for the triangular SUMMA schedule, where step i only reaches columns i to 3. The code sets the metric to $P - 1 = 3.0$ and the comment says the mean is $P - 1$. The hand-computed test expects 3.0, so the test passes and the error is invisible. The metric is the maximum hop count, or the broadcast depth, not the average. For a latency model the difference matters: the reported value is two to three times the mean.

This is a small bug with a clear fix: either rename the field to `broadcast_depth`, or compute the mean over the destination hops. The report records it because the same pattern appears elsewhere: a metric is defined by the code, the test checks the code's definition, and the name implies a stronger claim than the number supports.

8.4 The 99.99% fusion figure

The fusion traffic reduction is computed as

$$\frac{2 \times \text{intermediate elements}}{2 \times \text{intermediate elements} + \text{matmul wavelets}}.$$

The intermediate is the RMSNorm output, `tensor<1x2048x4096xbf16>`, with 8,388,608 elements. The model counts two wavelets per element, one write and one read, for 16,777,216 eliminated wavelets. The matmul broadcast is 2,048 wavelets. The ratio is 99.9878%.

The comparison mixes two scales. The numerator is the real fragment's intermediate, 8.4 million elements. The denominator's matmul term is a 32x32x32 toy with 2,048 wavelets. A real fragment matmul would inject far more than 2,048 wavelets, so the real ratio would be lower. The comparison also assumes that the unfused schedule sends the intermediate across the fabric. A schedule could keep the RMSNorm output in local SRAM and avoid the fabric entirely, in which case the eliminated traffic is zero and the ratio is undefined.

The model's own cross-reference says the two metrics are different views: the GPU traffic report is the whole-pipeline global-byte view, and the dataflow figure is the eliminated intermediate's fabric wavelets against the matmul broadcast. That is accurate as a description of the computation. It is not a performance result. The honest form is a conditional statement: if the unfused schedule materialises the intermediate and sends it

across the fabric, and if the matmul broadcast is the 32x32x32 toy, then the reduction is 99.99%. The repository's prose presents the number without the conditions.

The GPU traffic number is the better metric. Fusion eliminates 33,554,432 bytes from an analytic 478,150,656 bytes, a 7.02% reduction. Even that is an analytic minimum, not a measured DRAM count, and the measured benchmark shows that the eliminated traffic does not translate into time at a compute-bound shape.

8.5 What the simulator establishes

The simulator establishes four things.

1. The task model can express dataflow-triggered compute: tasks fire on wavelet arrival, and a blocked task does not fire early. The negative test covers this.
2. The SUMMA schedule can be lowered from the matmul operation and executed, and the output matches the golden for a 32x32x32 matmul.
3. The metrics are computed from the plan and the run, and the hand-computed test checks the arithmetic.
4. A fused operation can be represented as local PE compute with no fabric wavelets, which is the mechanism behind the 99.99% figure.

It does not establish timing fidelity to Cerebras hardware, performance at the fragment shape, or a comparison with a real CSL implementation. It is a teaching model with a working functional core, and the report treats it as such.

9 Serving

9.1 The Modal app

`modal/app.py` defines a class-based Modal app on an A10 GPU with four FastAPI endpoints:

Table 8. The Modal endpoint contract.

Endpoint	Method	Purpose
`/predict`	POST	Run the fused MLP path on input data
`/benchmark`	POST	Run the correctness-gated benchmark for an operation and shape
`/kernels`	GET	List the kernels in the cache for the detected GPU
`/report`	GET	Return the contract-H report for a kernel

The image is `nvidia/cuda:12.8.1-devel-ubuntu24.04` with Python 3.12. The class sets `min_containers=1`, a 300-second scaledown window, and memory snapshotting. Two `@modal.enter` hooks split the work: the snapshot-time hook loads the shared library and weights before the GPU is up, and the restore-time hook runs one dummy kernel warmup after the GPU is restored. The GPU-aware kernel cache reuses the C4 runtime's `DeviceDetect` and `KernelCache`.

The design is sensible for cold-start work. The split hook is the right pattern for Modal's memory snapshot, and the cache key includes the GPU, operation, and shape.

9.2 Status

The app is a Modal class-based service [13]. It was imported and constructed locally. The test confirmed that the class, the four endpoints, the request models, and the volume mount are valid without a token. `modal`

`serve` failed with a missing-token error. The deploy script degraded to SKIPPED, and `cold_start.json` records `status: "skipped-no-deployment"` with null cold and warm times.

No request was served. No cold-start measurement exists. The `/predict` path has no numerical test against the golden. The image was not built.

This is a clear case of an implemented contract with no runtime evidence. The report does not count it as a working serving path. The code is there, the design is documented, and the next step is a token and a deploy, not more code.

9.3 What would make it evidence

A deploy would need a Modal token and a GPU. The minimum evidence set is:

1. `modal deploy` succeeds and prints a URL.
2. `/kernels` returns the sm_89 or A10 entry from the cache.
3. `/predict` on a known input matches the golden within Contract C.
4. `/report` returns a contract-H JSON with the measured timing.
5. A cold-start run records a cold time above the warm time after a scale-down.

Until then, the endpoint table is a specification, not a result.

10 Engineering assessment

10.1 Build and reproducibility

The build is a devenv development shell [14] on a Nix flake for `x86_64-linux` only. It pins MLIR and LLVM 21.1.8 through `llvmPackages_21`, CUDA 12.6 through `cudaPackages_12_6`, ROCm through `rocmPackages` (7.2.3 on the author's host), Python 3 with NumPy, `ml_dtypes`, pytest, and Modal, and gtest. The project uses C++20 with `CMAKE_CXX_STANDARD 20` and `CXX_EXTENSIONS OFF`.

The build model is a development shell plus a manual CMake configure, not a sandboxed `nix build`. The reason is documented: the deliverable is the compiler and its tests, and a sandboxed build of the whole project would add packaging work without changing the artifact. The CMake configure needs `MLIR_DIR`, `LLVM_DIR`, `MLIR_TABLEGEN_EXE`, and `LLVM_EXTERNAL_LIT`. The compiler document gives a reproducible way to obtain the first two by querying the flake's own nixpkgs input, and a fragile way that hardcodes `/nix/store/...` paths from one evaluation. The reproducible way is the right one.

The README's quick start uses

```
nix eval --raw --impure --expr \
  '(builtins.getFlake "/home/mei/projects/polykernel").inputs.nixpkgs...'
```

That path is the author's home directory. On any other checkout the command fails. The fix is to use `(toString ./.)` or `builtins.getFlake (toString ./.)`, which resolves the current flake. This report records the bug; the rewritten README uses the portable form.

Three other reproducibility gaps are worth naming:

- The flake has no `aarch64-linux` or `aarch64-darwin` output. The project cannot be built on an Apple Silicon machine, including the machine that produced this report.
- There is no CI workflow. The 14 lit tests, 158 ctest cases, and pytest suites are run by hand. A push can break the build without anyone noticing until the next local run.
- There is no `LICENSE` file. A public repository without a license is not legally reusable, whatever the README says.

10.2 Documentation drift

The earlier documents are detailed and mostly accurate about the design. Several quantitative claims drifted from the code and the logs. Table 9 lists the ones that matter.

Table 9. Claims checked against source and evidence. Each row was verified by reading the named file or log.

Claim in the earlier prose	What the code or log shows	Status
"11 passes"	`Passes.td` declares 12	Wrong count
"11 base + 6 fused"	11 base + 5 fused + `func/return` = 18	Wrong count
"18/18 golden" on the Ada	17 golden cases plus 1 negative launch test	Imprecise
"HIP runs: 75/75 tests passed"	18 HIP tests; 75 is the whole `tests/kernels/` run	Wrong attribution
Dashboard "10/10 ops validated"	5 unfused + 4 fused + 1 cross-check on a CPU mini-MLP	Imprecise
"autotuned 0.857x"	Minimum of repeated runs of the same fused kernel	Misleading label
`matmul.cu` has "double-buffered smem" and `As[16][17]`	The emitted kernel has neither	Wrong
A100 80GB uses 1,555 GB/s	The 80GB part has 2,039 GB/s	Wrong constant
2.838x autotuned speedup	0.35 / 0.12 model ratio, no measured kernel	Projection, but headlined

The pattern is consistent: the design prose is accurate, and the numbers were written by hand after the fact. A generated table would not drift. The dashboard is generated and does carry the MEASURED and PROJECTED labels correctly, which is why the errors cluster in the hand-written README and docs, not in the JSON.

The 75/75 case is the clearest. The README says:

> HIP runs the shared portable template (scalar + WMMA bf16) golden-validated on the local RX 7800 XT: **75/75 tests passed**.

The committed HIP log says 18 passed. The 75 is the whole kernel test directory on the author's Linux host, which includes 18 HIP tests, 5 WMMA tests, 13 attention tests, 13 quantization tests, and a set of CPU-reference tests. The HIP-specific evidence is 18, not 75. The 75 is a real number, but it does not mean what the sentence says.

10.3 Test composition

The suite is broad for a project of this size. The 14 lit tests cover the compiler. The 158 ctest cases cover the analyzer, autotuner, dataflow, runtime, and cache. The pytest directory covers CPU references, HIP, attention, quantization, MMA, and WMMA. The negative tests are unusually good.

The gaps are equally clear. There is no test of the generated CUDA kernel at the benchmark shape, no test of the Modal endpoint, no H100 or MI300X run, and no test that the projected numbers match a measured rate on any GPU. The autotuner's correctness gate at the benchmark shape uses an infinite relative ceiling. The dataflow test uses a 32x32x32 shape.

A reader should treat the suite as evidence that the machinery works on the shapes it exercises, not as evidence that it works at production shapes.

10.4 Prepared upstream contribution

The repository also contains a prepared contribution to Triton, `triton-lang/triton`: a new tutorial, `python/tutorials/11-fused-matmul-bias-gelu.py`, about 172 lines. It reuses the tiled GEMM from tutorial 03 and adds a per-column bias and an exact erf GELU in the epilogue, while the accumulator is still fp32.

The status is prepared and not submitted. No fork was created, no pull request was opened, and no maintainer was contacted. The file was syntax-checked with `py_compile`; it was not run on a GPU. The tutorial's numerical convention differs from PolyKernel's golden: Triton keeps the epilogue in fp32 and casts once at the store, while PolyKernel rounds after each stage, so the two are not bit-identical. The tutorial claims correctness against a PyTorch reference at 1e-2 tolerance, not against the PolyKernel golden.

The contribution is a reasonable small patch and a poor piece of evidence for this report. It shows that the author can write a Triton tutorial, not that the project has an upstream footprint. The report counts it as an optional artifact, not as a result. The extracted file is kept at `report/src/triton-fused-matmul-bias-gelu.py` so the work survives the removal of `docs/`, with the same status note at the top of this subsection.

10.5 Repository hygiene

The repository has 75 commits, a clean working tree at the start of this report, and a remote on GitHub. The `.gitignore` excludes build directories, caches, Python bytecode, and `*.pem`. The key-hygiene check found no tracked private key, mode 600 on the local key, and no key material in tracked files. The evidence logs contain the pod's IP address and user name; those are historical and the pod has been terminated.

The commit messages follow the imperative style and explain the why. The history is readable. The project would benefit from a license and a CI workflow more than from more features.

11 Threats to validity

The report's conclusions are only as strong as the evidence behind them. Table 10 lists the main threats and how they were handled.

Table 10. Threats to validity.

Threat	Effect	Handling
No independent rebuild	The audit cannot confirm that the committed logs match a fresh build	Arithmetic was recomputed; source was read; the report states that it did not rebuild
One shape per benchmark	The 0.838x result may not hold at other shapes	The mechanism is explained from the source; the report claims only the measured shape
One GPU per vendor	Ada and RDNA3 results may not transfer to H100 or MI300X	The report does not transfer them; the projections are audited separately
Self-reported logs	A log can be edited or produced by a different binary	The logs include commands, device probes, and negative cases; the report cross-checks counts and JSON
Clock and power variation	Timings vary with thermals and clocks	Run-to-run medians vary by 1.2% to 3.9%; the 19.6% gap is outside that band
Small correctness shapes	Passing at 128x128 does not prove 2048x11008	The report states the shape limit; the benchmark shape is timed, not golden-checked

Threat	Effect	Handling
Infinite relative ceiling in the autotune	A high relative error can pass	The report records the 1953.125 value and treats the run as a cosine/PCC gate
Analytic traffic	The 7.02% is a lower bound, not a DRAM count	The report labels it analytic and contrasts it with the measured regression
Dataflow toy shape	32x32x32 does not predict fragment performance	The report treats the simulator as a functional model, not a performance predictor
Undeployed service	The serving path has no runtime evidence	The report counts it as a specification, not a working path

The largest threat is the absence of a fresh build. This report was produced on a machine that cannot run the toolchain. The audit re-derived every ratio and read the code, but it did not recompile the project or rerun a kernel. A reader who needs stronger evidence should run the reproduction commands in Section 12 on a Linux host with a GPU.

The second largest threat is the gap between the measured shape and the projected shapes. The measured 25.52 ms is for a 2048x11008x4096 matmul on an Ada GPU. The H100 projection is for the same shape, but the model's rate is not derived from the measurement. The report treats the projection as a scenario, not an extrapolation.

12 Reproduction

12.1 Requirements

- Linux x86_64. The flake is pinned to that system.
- Nix with flakes enabled.
- About 8 GB of disk for the development shell, the MLIR and CUDA closures, and the build.
- An NVIDIA GPU for the CUDA runtime tests, an AMD GPU for the HIP tests, or both. The compile-only paths need neither.

12.2 Build

```
nix develop --impure --accept-flake-config

# Resolve the locked MLIR and LLVM from this checkout, not a hardcoded path.
MLIR_DIR="$(nix eval --raw --impure --expr \
  '(builtins.getFlake (toString ./.)).inputs.nixpkgs.legacyPackages.x86_64-linux.llvmPackages_21.mlir.dev.c
LLVM_DIR="$(nix eval --raw --impure --expr \
  '(builtins.getFlake (toString ./.)).inputs.nixpkgs.legacyPackages.x86_64-linux.llvmPackages_21.llvm.dev.c

cmake -B build -G Ninja \
  -DMLIR_DIR="$MLIR_DIR" \
  -DLLVM_DIR="$LLVM_DIR" \
  -DMLIR_TABLEGEN_EXE="$(which mlir-tblgen)" \
  -DLLVM_EXTERNAL_LIT="$(dirname "$(readlink -f "$(which lit)")")/.lit-wrapped"
cmake --build build
cmake --build build --target check-polykernel
ctest --test-dir build --output-on-failure
```

The expected results are 14 of 14 lit tests and 158 of 158 ctest cases on the author's host. The numbers on another host may differ if a test depends on the GPU or the driver.

12.3 GPU tests

```
# AMD, gfx1101
python3 -m pytest tests/kernels/test_hip_run.py tests/kernels/test_wmma.py -q

# NVIDIA, sm_89 or another supported arch
python3 -m pytest tests/kernels/test_cuda_run.py tests/kernels/test_mma.py -q
```

The CUDA tests skip with a named reason when no device is present. The HIP tests need a working ROCm runtime and a gfx1101 device.

12.4 Benchmark

```
python3 tools/polykernel-bench/bench.py \
  --backend cuda --arch sm_89 --autotune \
  --op fused_matmul_bias_gelu --shape 2048,4096,11008 \
  --dtype bf16 --variants 8 --cache-out cache/tuning_cache.json
```

The C++ `benchmarks/bench_cuda.cpp` is the headline timing harness. It needs the pod or a local NVIDIA GPU and the generated kernels linked in. The `benchmarks/rent_runpod.sh` script provisions the rental path behind a spend ceiling.

12.5 Figures and PDF

The figures are generated from `report/figures/gen_figures.py` and rendered with the Excalidraw CLI:

```
EXCALIDRAW=/path/to/excalidraw \
EXCALIDRAW_BROWSER=/path/to/chrome \
python3 report/figures/gen_figures.py --render
```

The PDF is built with pandoc and XeLaTeX:

```
pandoc report/src/polykernel-report.md \
  -o report/polykernel-technical-report.pdf \
  --pdf-engine=xelatex --toc --number-sections \
  -V geometry:margin=2.4cm -V fontsize=10pt \
  -V mainfont="Palatino" -V monofont="Menlo" \
  -V colorlinks=true -V linkcolor=blue \
  --include-in-header=report/src/header.tex
```

12.6 What runs without a GPU

The lit suite, the ctest suite, the CPU-reference pytest cases, the traffic report, the dataflow simulator, and the dashboard do not need a GPU. The CUDA and HIP runtime tests skip without a device. The benchmark and the Modal service need hardware and, for Modal, a token.

13 Conclusion

PolyKernel is a coherent vertical slice of compiler and runtime machinery. The dialect is small and closed, the pass pipeline is readable, the source emitter is a deliberate trade that buys portability, and the shared template is what makes the two-vendor path tractable. The correctness harness is credible: it found two real CUDA bugs on the first NVIDIA run, it has negative tests for the gates, and it keeps the measured regression in the dashboard rather than hiding it.

The performance layer has not caught up with the compiler layer. The only measured GPU benchmark is a scalar prologue that fusion makes slower. The autotuner searches eight similar configurations and finds a 1.3% spread. The H100, A100, and MI300X numbers are the same roofline model with a changed attainment

constant, and the H100 and A100 variants exceed the scalar FP32 peak of the hardware they name. The 2.84x headline is the ratio 0.35/0.12, not a measured speedup.

The repository's labels are honest. The H100, A100, and MI300X rows say PROJECTED, the Ada row says MEASURED, and the performance document explains the attainment factors. The problem is emphasis and arithmetic, not intent. A reader who stops at the README table will remember the three large numbers and not the label.

Three changes would move the performance claim from scenario to evidence.

1. Compute the row RMS once per row in the fused kernel, then reuse it across the 688 output tiles. The current kernel recomputes it, and the measurement shows the cost. A fixed fused kernel would be a real test of whether prologue fusion helps at this shape.
2. Benchmark the MMA kernel at 2048x11008x4096 and use that measurement as the tensor-core anchor for the projection. The kernel exists and passes correctness; it has not been timed at the headline shape.
3. Run the benchmark on one H100. The harness, the rental script, and the generated code are ready. One measured point would replace three modelled rows.

Until then, the fair summary is this: PolyKernel is a working compiler and a credible correctness story with a performance story that has not been measured yet. That is a real artifact, and the report treats it as one.

Appendix A: evidence index

Table 11 maps the report's claims to the test or file that supports them. The prose and log files under `reports/` were removed when this report replaced them; the machine-readable inputs moved to `report/data/`, and the test names and commit hashes remain in the repository history.

Table 11. Evidence index. "Reported" means the number is in the committed log but was not independently reproduced here.

Claim	Evidence	Commit or file	Status
18 dialect operations	`PolyKernelOps.td`; `op_set_closed.mlir`	`include/PolyKernel/IR/`	Verified
12 passes	`Passes.td`	`include/PolyKernel/Passes/`	Verified
14 lit tests	lit output in the F3 log	`test/`	Verified
158 ctest cases	ctest output in the F3 log	`build/`	Reported
17 CUDA golden cases	`pass2_ada_cuda_run.json`; per-case table	`3f9b760`, `d85866a`	Verified
18 HIP cases	`w4_hip_correctness.log`	`reports/` (removed)	Verified
5 MMA cases	`pass2_ada_mma.log`	`9d7fea4`	Verified
Two CUDA bugs fixed	commit message and diff	`d85866a`	Verified
Ada medians and speedups	`ada6000_bench.json`	`c3bc7ad`	Verified
8 autotune configs	`pass2_ada_autotune.log`	`745e73b`	Verified
32 MiB traffic saving	`mlp_traffic.json`	`report/data/`	Verified arithmetic
7.02% traffic reduction	`mlp_traffic.json`	`report/data/`	Verified arithmetic
Dataflow metrics	`dataflow_metrics.json`	`report/data/`	Verified arithmetic
Projection constants	`h100_bench.json`, `mi300_bench.json`	`report/data/`	Verified arithmetic
Projection rate exceeds H100 FP32 peak	recomputation from the JSON	this report	Verified
Modal not deployed	`cold_start.json`	`report/data/`	Verified

Appendix B: recomputation

The following values are recomputed from the raw data.

- Matmul flops: $2 \times 2048 \times 11008 \times 4096 = 184{,}683{,}593{,}728$.
- Unfused rate: $184.6836 / 0.025518 = 7.237$ TFLOP/s.
- Fused rate: $184.6836 / 0.030459 = 6.063$ TFLOP/s.
- Autotuned minimum rate: $184.6836 / 0.029785 = 6.201$ TFLOP/s.
- Fusion penalty: $25.518 / 30.459 - 1 = -19.6\%$.
- Ada fraction of peak: $7.237 / 91.1 = 7.94\%$.
- Redundant RMS reads: $88{,}064 \times 16 \times 4096 \times 2 = 11{,}542{,}724{,}608$ bytes.
- Saving-to-redundancy ratio: $33{,}554{,}432 / 11{,}542{,}724{,}608 = 0.29\%$.
- Traffic reduction: $33{,}554{,}432 / 478{,}150{,}656 = 7.0175\%$.
- H100 implied rate: $184.6836 / 0.001556 = 118.68$ TFLOP/s.
- H100 ratio to scalar peak: $118.68 / 66.9 = 1.77$.
- H100 time at 12% of scalar peak: $184.6836 / (0.12 \times 66.9) = 23.0$ ms.
- A100 implied rate: $184.6836 / 0.004933 = 37.44$ TFLOP/s.
- A100 ratio to scalar peak: $37.44 / 19.5 = 1.92$.
- MI300X implied rate: $184.6836 / 0.001177 = 156.9$ TFLOP/s.
- Attainment ratio: $0.35 / 0.12 = 2.917$.
- Fragment ratio: $3.17724 / 1.1195 = 2.838$.
- Dataflow eliminated wavelets: $2 \times 8{,}388{,}608 = 16{,}777{,}216$.
- Dataflow ratio: $16{,}777{,}216 / (16{,}777{,}216 + 2{,}048) = 99.9878\%$.

References

[1] NVIDIA. *H100 Tensor Core GPU*. Product page and datasheet, accessed 31 October 2026. https://www.nvidia.com/en-us/data-center/h100/

[2] NVIDIA. *A100 Tensor Core GPU*. Product page and datasheet, accessed 31 October 2026. https://www.nvidia.com/en-us/data-center/a100/

[3] NVIDIA. *RTX 6000 Ada Generation*. Product page, accessed 31 October 2026. https://www.nvidia.com/en-us/design-visualization/rtx-6000/

[4] AMD. *AMD Instinct MI300X Accelerators*. Product page, accessed 31 October 2026. https://www.amd.com/en/products/accelerators/instinct/mi300/mi300x.html

[5] NVIDIA. *CUDA C++ Programming Guide*, arithmetic instructions and native arithmetic throughput table. https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#arithmetic-instructions

[6] LLVM Project. *MLIR: Defining Dialects*. https://mlir.llvm.org/docs/DefiningDialects/

[7] S. Williams, A. Waterman, and D. Patterson. "Roofline: An Insightful Visual Performance Model for Multicore Architectures." *Communications of the ACM*, 52(4), 2009. https://doi.org/10.1145/1498765.1498785

[8] Cerebras. *Cerebras SDK: CSL documentation*, wavelets, colors, routes, and fabric DSDs. https://sdk.cerebras.net/csl/

[9] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." *NeurIPS*, 2022. https://arxiv.org/abs/2205.14135

[10] P. Tillet, H. Kung, and D. Cox. "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations." *MAPL*, 2019. https://doi.org/10.1145/3315508.3329973

[11] H. Elshafie. "Worklog: Optimising GEMM on NVIDIA H100 for cuBLAS-like Performance (WIP)." January 2026. https://hamzaelshafie.bearblog.dev/worklog-optimising-gemm-on-nvidia-h100-for-cublas-like-performance-wip/

[12] S. Boehm. "How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance: a Worklog." 2022. https://siboehm.com/articles/22/CUDA-MMM

[13] Modal. *Modal documentation*, class-based apps, memory snapshot, and FastAPI endpoints. https://modal.com/docs

[14] devenv. *devenv documentation*, Nix development shells. https://devenv.sh/